DIGITAL DESIGN & COMPUTER ORGANIZATION

SUB CODE :- BCS302

Lab Manual for 3rd Sem

AMC EDUCATION



VISION

"Be a premier department in the field of Computer Science & Engineering to meet the technological challenges of the society".

Mission of the Department

M1	To provide the State-of-the-art Infrastructure Facilities
M2	To provide the exposure to the latest tools in the area of Computer Hardware and Software.
M3	To strive for Academic excellence through Research in Computer Science and Engineering with creative Teaching – Learning pedagogy
M4	To establish Industry-Institute Interaction and make Students ready for the Industrial environment.
M5	To transform Students into Entrepreneurial, Technically competent, Socially responsible and Ethical computer science professional.

Program Educational Objectives (PEOs) of Department

After course completion CSE graduates will be able to:

PEO 1	Graduates acquire advanced knowledge of Computer Science Engineering and excel in leadership roles to serve the society.
PEO 2	Graduates of the program will apply Computer Science and Engineering and excel in leadership computer science professional.
PEO 3	Graduates adapt Value-Based Proficiency in solving real time problems.

Program Specific Outcomes (PSO)

PSO 1	**Professional Skills:** Ability of applying the Computing Concepts, Data Structure, Computer Networks and Suitable Algorithm.
PSO 2	**Software Skills:** Ability to build Software Engineering System with Development Life Cycle by using analytical knowledge in Computer Science and Engineering and applying modern methodologies

Program Outcomes (POs)

PO1	**Engineering knowledge**: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
PO2	**Problem analysis**: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO3	**Design/development of solutions**: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
PO4	**Conduct investigations of complex problems**: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions
PO5	**Modern tool usage**: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
PO6	**The engineer and society**: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
PO7	**Environment and sustainability**: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
PO8	**Ethics**: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
PO9	**Individual and team work**: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
PO10	**Communication**: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO11	**Project management and finance**: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO12	**Life-long learning**: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

AMC ENGINEERING COLLEGE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Course Details

Digital Design and Computer Organization

Subject Code: BCS302

Course objectives:

- To demonstrate the functionalities of binary logic system
- To explain the working of combinational and sequential logic system
- To realize the basic structure of computer system
- To illustrate the working of I/O operations and processing unit.

CIE for the practical component of the IPCC

- 15 marks for the conduction of the experiment and preparation of laboratory record, and 10 marks for the test to be conducted after the completion of all the laboratory sessions.
- On completion of every experiment/program in the laboratory, the students shall be evaluated Including viva-voce and marks shall be awarded on the same day.
- The CIE marks awarded in the case of the Practical component shall be based on the continuous evaluation of the laboratory report. Each experiment report can be evaluated for 10 marks. Marks of all experiments' write-ups are added and scaled down to 15 marks.
- The laboratory test (duration 02/03 hours) after completion of all the experiments shall be conducted for 50 marks and scaled down to 10 marks.

● Scaled-down marks of write-up evaluations and tests added will be CIE marks for the laboratory component of IPCC for 25 marks.

● The student has to secure 40% of 25 marks to qualify in the CIE of the practical component of the IPCC.

Experiments

Simulation packages preferred: Multisim, Modelsim, PSpice or any other relevant

	Simulation packages preferred: Multisim, Modelsim, PSpice or any other relevant
1	Given a 4-variable logic expression, simplify it using appropriate technique and simulate the same using basic gates.
2	Design a 4 bit full adder and subtractor and simulate the same using basic gates.
3	Design Verilog HDL to implement simple circuits using structural, Data flow and Behavioural model.
4	Design Verilog HDL to implement Binary Adder-Subtractor – Half and Full Adder, Half and Full Subtractor.
5	Design Verilog HDL to implement Decimal adder
6	Design Verilog program to implement Different types of multiplexer like 2:1, 4:1 and 8:1.
7	Design Verilog program to implement types of De-Multiplexer.
8	Design Verilog program for implementing various types of Flip-Flops such as SR, J and D.

Course Outcome

CO1	Apply the K–Map techniques to simplify various Boolean expressions
CO2	Design different types of combinational and sequential circuits along with Verilog programs
CO3	Describe the fundamentals of machine instructions, addressing modes and Processor performance
CO4	Explain the approaches involved in achieving communication between processor and I/O devices
CO5	Analyze internal Organization of Memory and Impact of cache/Pipelining on Processor Performance.

No	Programs	CO
1	Given a 4-variable logic expression, simplify it using appropriate technique and simulate the same using basic gates.	**CO1**
2	Design a 4 bit full adder and subtractor and simulate the same using basic gates.	**CO1**
3	Design Verilog HDL to implement simple circuits using structural, Data flow and Behavioural model.	**CO1**
4	Design Verilog HDL to implement Binary Adder-Subtractor – Half and Full Adder, Half and Full Subtractor.	**CO1**
5	Design Verilog HDL to implement Decimal adder	**CO1**
6	Design Verilog program to implement Different types of multiplexer like 2:1, 4:1 and 8:1.	**CO2**
7	Design Verilog program to implement types of De-Multiplexer.	**CO2**
8	Design Verilog program for implementing various types of Flip-Flops such as SR, J and D.	**CO2**

TABLE OF CONTENTS

PRACTICAL COMPONENT OF IPCC

Sl.NO	Experiments Simulation packages preferred: Multisim, Modelsim, PSpice or any other relevant
1	Given a 4-variable logic expression, simplify it using appropriate technique and simulate the same using basic gates.
2	Design a 4 bit full adder and subtractor and simulate the same using basic gates.
3	Design Verilog HDL to implement simple circuits using structural, Data flow and Behavioural model.
4	Design Verilog HDL to implement Binary Adder-Subtractor – Half and Full Adder, Half and Full Subtractor.
5	Design Verilog HDL to implement Decimal adder.
6	Design Verilog program to implement Different types of multiplexer like 2:1, 4:1 and 8:1.
7	Design Verilog program to implement types of De-Multiplexer.
8	Design Verilog program for implementing various types of Flip-Flops such as SR, JK and D.

Program 1

Given a 4-variable logic expression, simplify it using appropriate using basic gates.

4-variable Boolean Expression - aʾd ʾ+ bʾc ʾ+ bʾd ʾ+ ad

Truth Table :

a	b	c	d	a'd' + b'c' + b'd' + ad
0	0	0	0	
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	1
0	1	1	1	0
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1

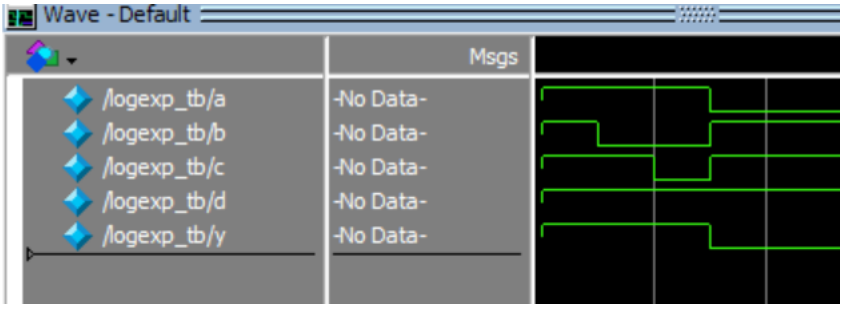
Verilog code:

```
module logexp(a,b,c,d,y);
input a,b,c,d;
output y;
reg y;
always @(a,b,c,d)
begin
y=(~a&~d)|(~b&~c)|(~b&~d)|(a&d);
end
endmodule
```

Testbench code :

```
module logexp_tb;
reg a,b,c,d;
wire y;
logexp logexp_tb(a,b,c,d,y);
initial
begin
a=1;b=1;c=1;d=1;
#10 a=1;b=0;c=1;d=1;
#10 a=1;b=0;c=0;d=1;
#10 a=0;b=1;c=1;d=1;
end
endmodule
```

Simulation Output :



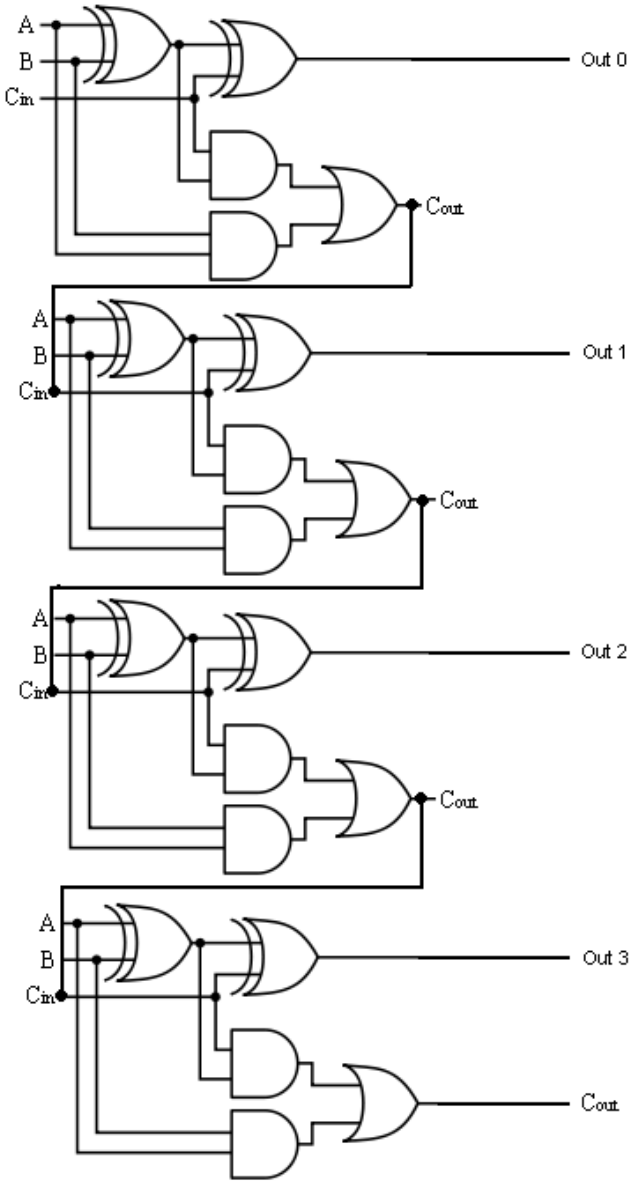
Program 2

Design a 4-bit full adder and Subtraction and simulate the same using basic gates.

Truth Table :

	A				B				Sum				Carry
Cin	A3	A2	A1	A0	B3	B2	B1	B0	S3	S2	S1	S0	Cout
0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	1	0	0	1	0	0
0	0	0	1	0	0	0	1	0	0	1	0	0	0
0	0	0	1	1	0	0	1	1	0	1	1	0	0
0	0	1	0	0	0	1	0	0	1	0	0	0	0
0	0	1	0	1	0	1	0	1	1	0	1	0	0
0	0	1	1	0	0	1	1	0	1	1	0	0	0
0	0	1	1	1	0	1	1	1	1	1	1	0	0
0	1	0	0	0	1	0	0	0	0	0	0	0	1
0	1	0	0	1	1	0	0	1	0	0	1	0	1
0	1	0	1	0	1	0	1	0	0	1	0	0	1
0	1	0	1	1	1	0	1	1	0	1	1	0	1
0	1	1	0	0	1	1	0	0	1	0	0	0	1
0	1	1	0	1	1	1	0	1	1	0	1	0	1
0	1	1	1	0	1	1	1	0	1	1	0	0	1
0	1	1	1	1	1	1	1	1	1	1	1	0	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1

Logic Diagram :



Verilog code:

```
  module add_sub_4(A, B, In, Res, Out);
 input [3:0] A, B;
 input In;
 output [3:0] Res;
 output Out;
 wire t0,t1,t2,t3,t4,t5,t6;
xor x3(t0,B[0],In);
 xor x4(t1,B[1],In);
 xor x5(t2,B[2],In);
 xor x6(t3,B[3],In);
 fadr f5(A[0],t0,In,Res[0],t4);
 fadr f6(A[1],t1,t4,Res[1],t5);
 fadr f7(A[2],t2,t5,Res[2],t6);
 fadr f8(A[3],t3,t6,Res[3],Out);
endmodule

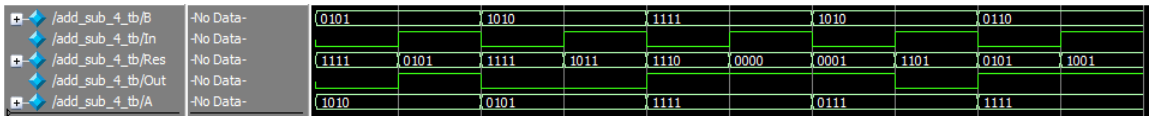
 module fadr(A, B, Cin, Sum, Cout);
 input A, B;
 input Cin;
 output Sum;
 output Cout;
 wire t1,t2,t3,t4;
 xor x1(t1,A,B);
 xor x2(Sum,t1,Cin);
 and g1(t2,A,B);
 and g2(t3,B,Cin);
 and g3(t4,Cin,A);
 or g4(Cout,t2,t3,t4);
 endmodule
```

Testbench code :

```
module add_sub_4_tb;
reg [3:0] A,B;
reg In;
wire [3:0] Res;
wire Out;
add_sub_4 add_sub_4_tb(A,B,In,Res,Out);
```

```
initial
begin
In=0; A=4'b1010; B=4'b0101;
#10 In=1;
#10 In=0; A=4'b0101; B=4'b1010;
#10 In=1;
#10 In=0; A=4'b1111; B=4'b1111;
#10 In=1;
#10 In=0; A=4'b0111; B=4'b1010;
#10 In=1;
#10 In=0; A=4'b1111; B=4'b0110;
#10 In=1;
end
endmodule
```

Simulation Output :



Program 3

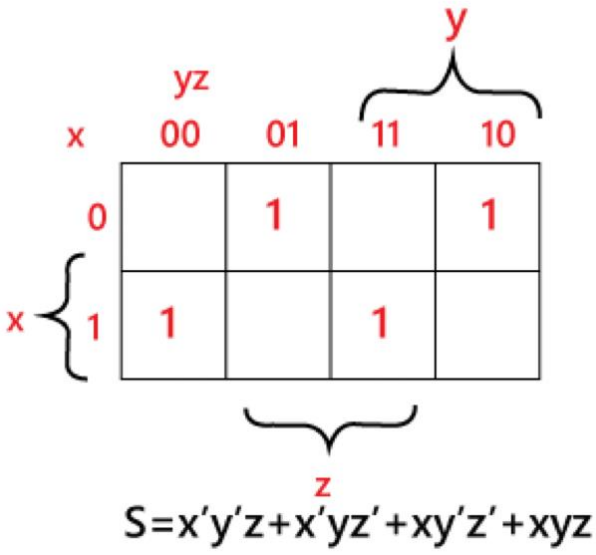
Design Verilog code to implement simple circuits using structural, Data flow, and Behavioural model.

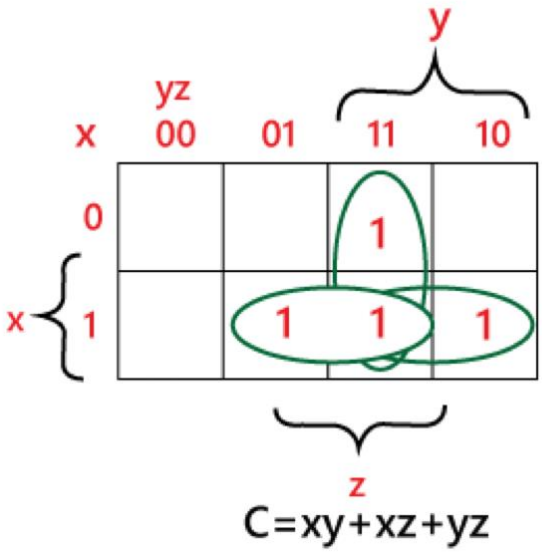
FULL ADDER

Truth Table :

Inputs			Outputs	
A	B	C_{in}	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Boolean Expression :





STRUCTURAL MODEL : VERILOG Code

```
module faddr(a,b,cin, s,c);
input a,b,cin;
output s,c;
xor(s1,a,b);
xor(s,s1,cin);
and (s2,a,b);
 and(s3,b,cin);
and(s4,a,cin);
or(c,s2,s3,s4);
endmodule
```

DATA FLOW MODEL : VERILOG Code

```
module faddr(a,b,cin, s,);
input a,b,cin;
output s,cout;
assign s = a ^ b ^ cin;
assign c = (a & b) | (b & cin) | (cin & a);
endmodule
```

BEHAVIORAL MODULE : VERILOG Code

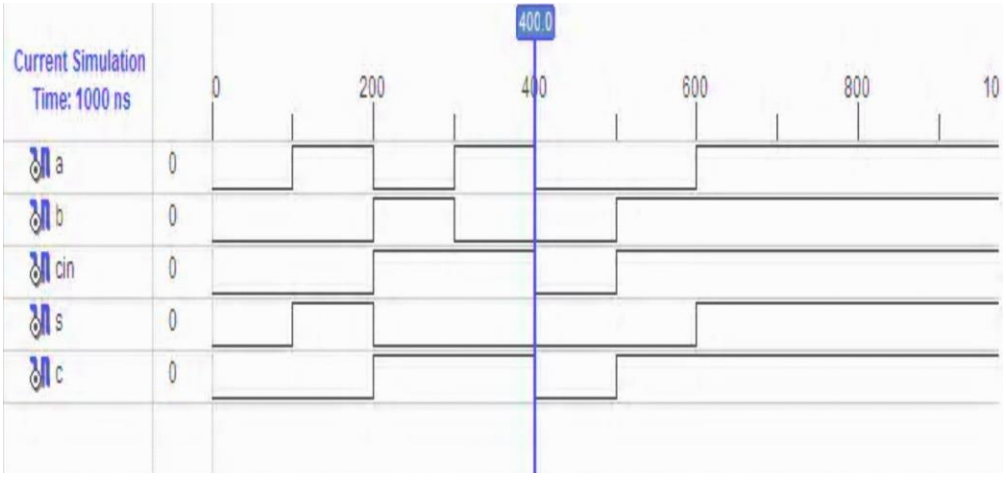
```
module faddr(a,b,cin, s,c);
input a,b,cin;
output s,c;
reg s,c;
always @ (a,b,cin)
begin
s= a ^ b ^ cin;
c = (a & b) | (b & cin) | (cin & a);
end
endmodule
```

Testbench code

```
module faddr_tb;
reg a,b,cin;
wire s,c;
faddr faddr_tb(a,b,cin,s,c);
initial
begin
a=0;b=0;cin=0;
#15 a=0;b=1;cin=0;
#15 a=1;b=0;cin=0;
#15 a=1;b=1;cin=0;
#15 a=0;b=0;cin=1;
#15 a=0;b=1;cin=1;
#15 a=1;b=0;cin=1;
#15 a=1;b=1;cin=1;
```

end
endmodule

Simulation Output :



Program 4

Design Verilog HDL to implement Binary adder-subtractor: Half and Full adder, Half and Full Subtractor.

Half Adder :

Truth Table

Inputs		Outputs	
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Boolean Expression :

S = a'b+ab'
C = ab

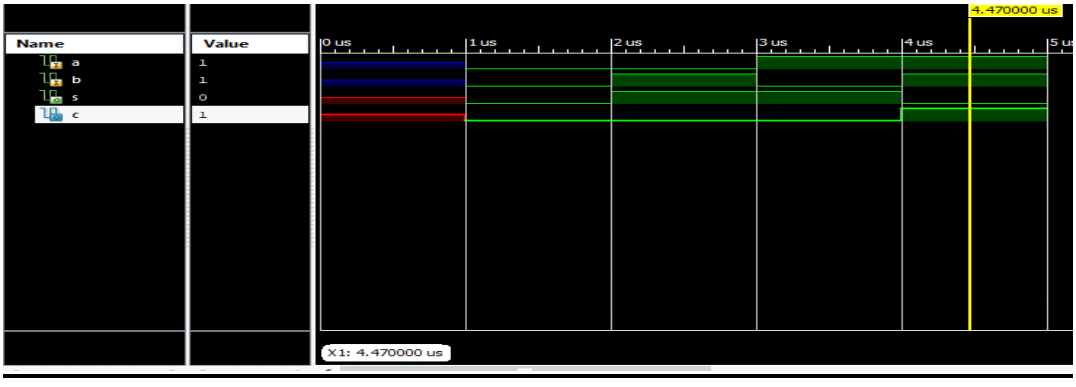
Half-Adder – Verilog code

```
module half_adder (a,b,s,c);
input a,b;
output s,c;
assign s = a^b;
assign c = a&b;
endmodule
```

Testbench code

```
module half_adder_tb;
reg a,b;
wire s,c;
half_adder half_adder_tb(a,b,s,c);
initial
begin
a=0;b=0;
#15 a=0;b=1;
#15 a=1;b=0;
#15 a=1;b=1;
end
endmodule
```

Simulation Output :



Half-Subtractor :

Truth Table

Inputs		Outputs	
A	B	Diff	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Boolean Expression :

Diff= A'B+AB'
Borrow = A'B

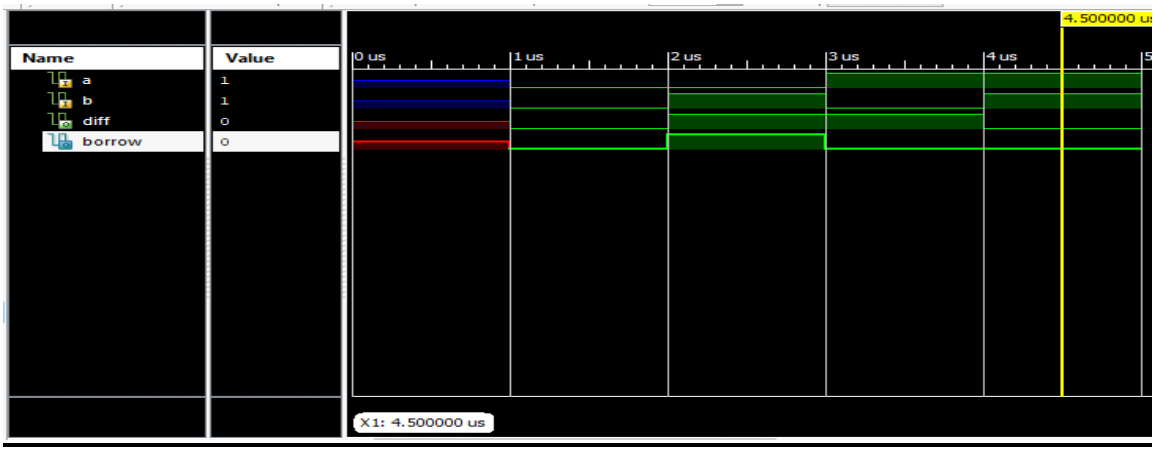
Half-Subtractor – Verilog Code :

```
module half_subtr (a,b, d,bw);
input a,b;
output d,bw;
d= a ^ b;
bw = ~a & b;
end
endmodule
```

Testbench code

```
module half_subtr _tb;
reg a,b;
wire d,bw;
half_ subtr  half_ subtr _tb(a,b,d,bw);
initial
begin
a=0;b=0;
#15 a=0;b=1;
#15 a=1;b=0;
#15 a=1;b=1;
end
endmodule
```

Simulation Output :

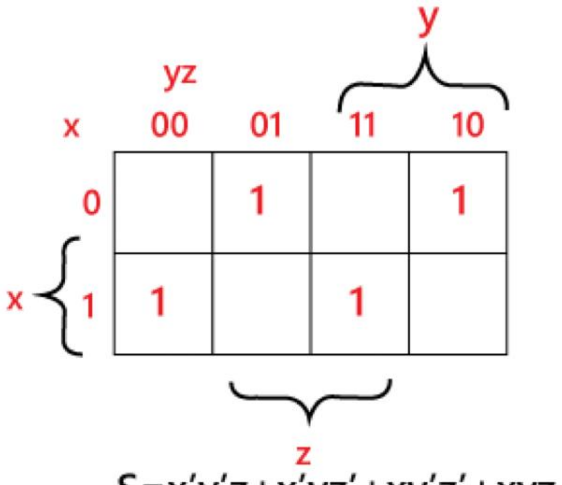


FULL ADDER :

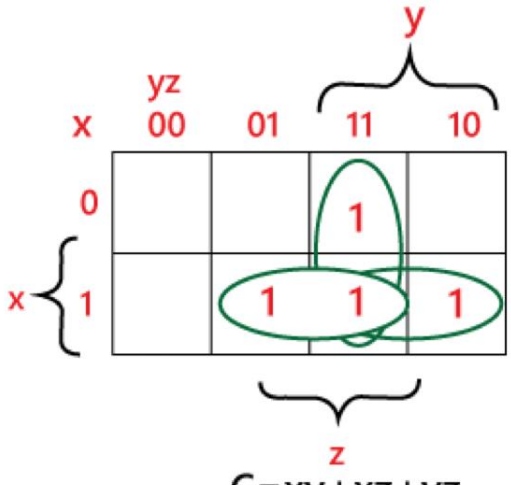
Truth Table:

Inputs			Outputs	
A	B	C_{in}	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Boolean Expression :



$S = x'y'z + x'yz' + xy'z' + xyz$



$C = xy + xz + yz$

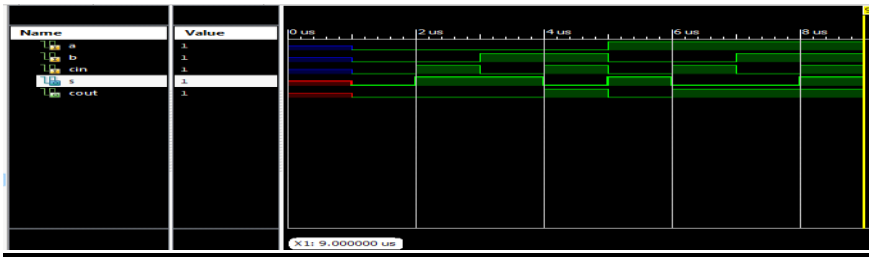
Full Adder Verilog Code :

```
module faddr(a,b,cin, s,cout)
input a,b,cin;
output s,cout;
assign s = a ^ b ^ cin;
assign cout = (a & b) | (b & cin) | (cin & a);
endmodule
```

Testbench code :

```
module faddr_tb;
reg a,b,cin;
wire s,cout;
faddr faddr_tb(a,b,cin,s,cout);
initial
begin
a=0;b=0;cin=0;
#15 a=0;b=1;cin=0;
#15 a=1;b=0;cin=0;
#15 a=1;b=1;cin=0;
#15 a=0;b=0;cin=1;
#15 a=0;b=1;cin=1;
#15 a=1;b=0;cin=1;
#15 a=1;b=1;cin=1;
end
endmodule
```

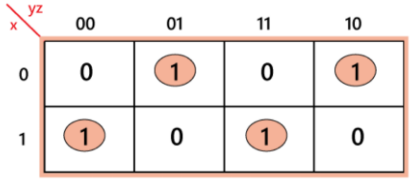
Simulation Output :



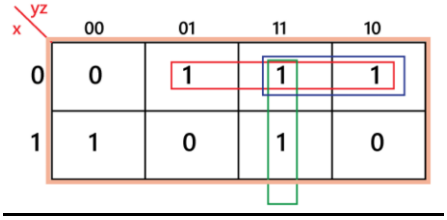
Full subtractor

Truth Table:

Inputs			Outputs	
A	B	$Borrow_{in}$	Diff	Borrow
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1



Diff=xy' z'+x' y' z+xyz+x'yz'



Borrow=x' z+x' y+yz

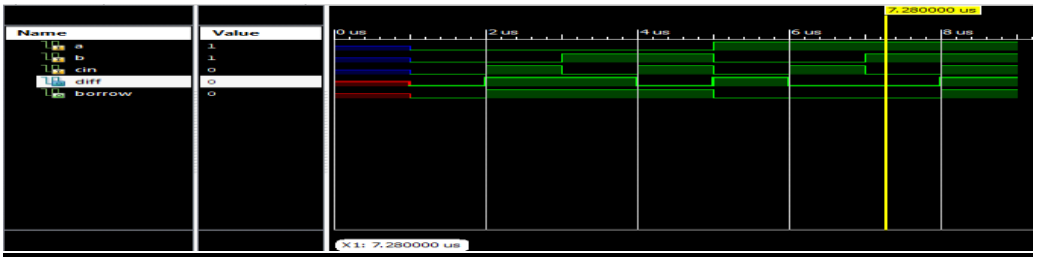
Full subtractor - Verilog Code

```
module fsubr(a,b,c,d,bw);
input a,b,c;
output d,bw;
assign d = a^b^c;
assign bw = (~a&c)|(~a&b)|(b&c);
endmodule
```

Testbench code :

```
module fsubr_tb;
reg a,b,c;
wire d,bw;
fsubr fsubr_tb(a,b,c,d,bw);
initial
begin
a=0;b=0;c=0;
#15 a=0;b=1;c=0;
#15 a=1;b=0;c=0;
#15 a=1;b=1;c=0;
#15 a=0;b=0;c=1;
#15 a=0;b=1;c=1;
#15 a=1;b=0;c=1;
#15 a=1;b=1;c=1;
end
endmodule
```

Simulation Output :

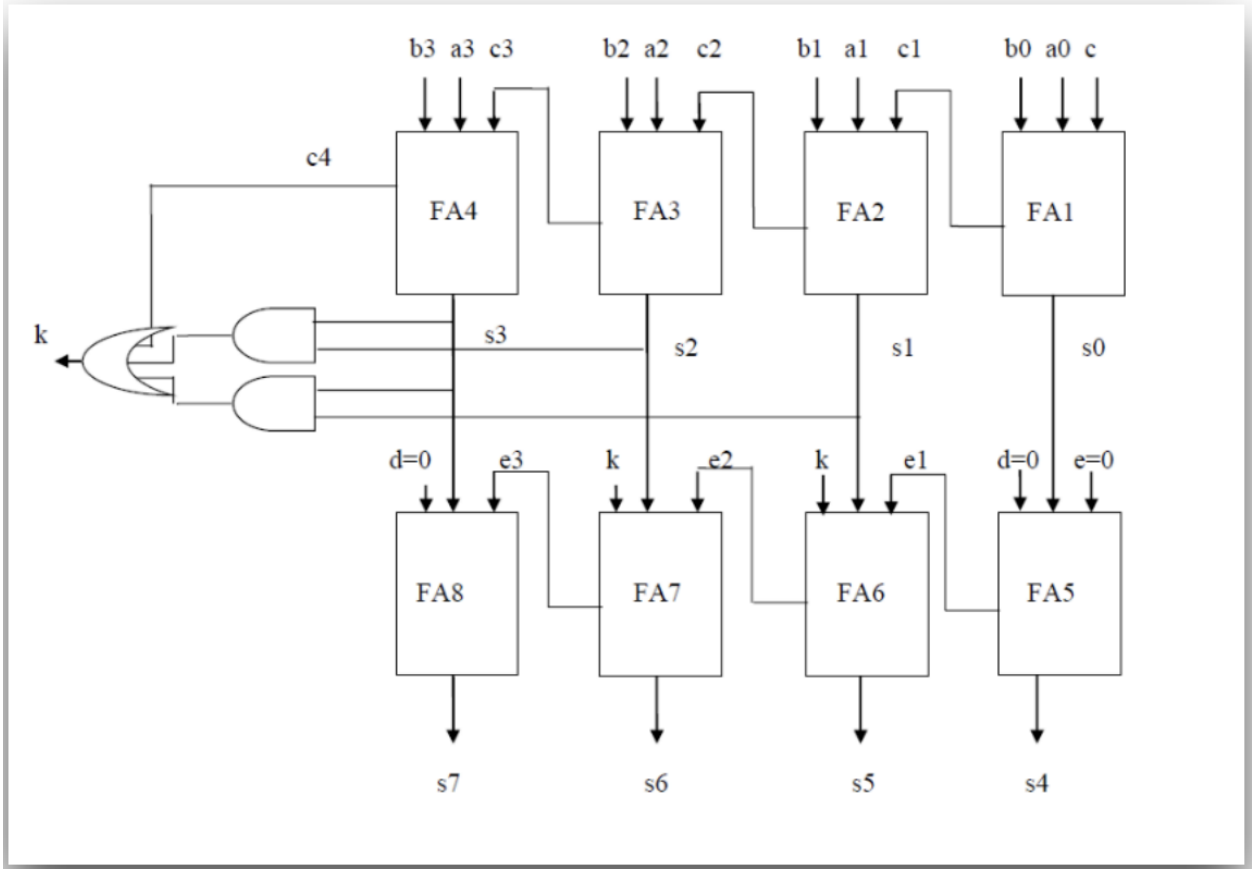


Program 5

Design Verilog HDL to implement Decimal adder.

Decimal					BCD Code					
K	Z_8	Z_4	Z_2	Z_1	C	S_8	S_4	S_2	S_1	**Decimal**
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	2
0	0	0	1	1	0	0	0	1	1	3
0	0	1	0	0	0	0	1	0	0	4
0	0	1	0	1	0	0	1	0	1	5
0	0	1	1	0	0	0	1	1	0	6
0	0	1	1	1	0	0	1	1	1	7
0	1	0	0	0	0	1	0	0	0	8
0	1	0	0	1	0	1	0	0	1	9
0	1	0	1	0	1	0	0	0	0	10
0	1	0	1	1	1	0	0	0	1	11
0	1	1	0	0	1	0	0	1	0	12
0	1	1	0	1	1	0	0	1	1	13
0	1	1	1	0	1	0	1	0	0	14
0	1	1	1	1	1	0	1	0	1	15
1	0	0	0	0	1	0	1	1	0	16
1	0	0	0	1	1	0	1	1	1	17
1	0	0	1	0	1	1	0	0	0	18
1	0	0	1	1	1	1	0	0	1	19

Logic Diagram :



A BCD adder, takes in two BCD numbers as inputs, and outputs a BCD digit along with a carry output. If the sum of the BCD digits is less than or equal to 9, then we don't have a problem. But if its greater than 9, we have to convert it into BCD format. This is done by adding 6 to the sum and taking only the least significant 4 bits. The MSB bit is output as carry.

Consider the below BCD addition :
1001 + 1000 = 10001
  9 + 8 = 17

The output 17 is more than 9. So we add 6 to it. So we get,
17+6 = 23 (in binary 23 is 10111)
Now the least significant 4 bits(which is "0111") represent the units digit and the MSB(4th bit which is '1') bit represents the tens digit.

Since the range of input is 0 to 9, the maximum output is 18. If you consider a carry it becomes 19. This means at the output side we need a 4 bit sum and a 1 bit carry to represent the most significant digit.

Verilog code for BCD (Decimal) adder:

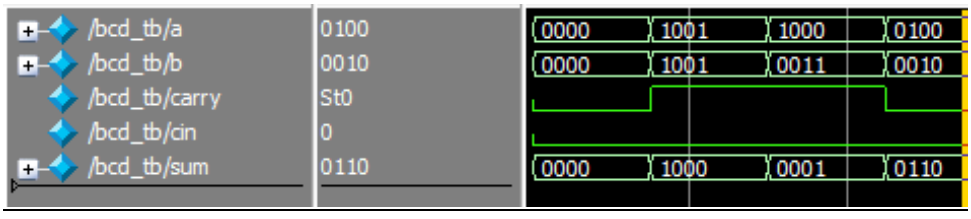
```
module bcd(a,b,cin,sum,carry);
input [3:0] a,b;
input cin;
output [3:0] sum;
output carry;
reg [4:0] sum_temp;
reg [3:0] sum;
reg carry;
always @(a,b,cin)
begin
 sum_temp = a+b+cin; //add all the inputs
if(sum_temp > 9) begin
 sum_temp = sum_temp+6; //add 6, if result is more than 9.
 carry = 1; //set the carry output
 sum = sum_temp[3:0]; end
else begin
 carry = 0;
 sum = sum_temp[3:0];
end
end
```

```
endmodule
```

Testbench for BCD adder:

```
module bcd_tb;
reg [3:0] a;
reg [3:0] b;
reg cin;
wire [3:0] sum;
wire carry;
bcd bcd_tb(a,b,cin,sum,carry);
initial
begin
a[3:0] = 4'b0000;b = 4'b0000;cin = 1'b0;
#15 a[3:0] = 4'b1001;b = 4'b1001;cin = 1'b0;
#15 a[3:0] = 4'b1000;b = 4'b0011;cin = 1'b0;
#15 a[3:0] = 4'b0100;b = 4'b0010;cin = 1'b0;
end
endmodule
```

Simulation waveform:



Program 6

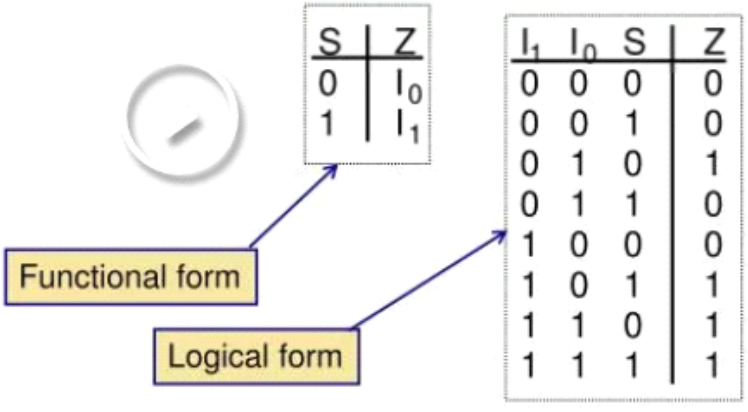
Design Verilog HDL to implement different types of multiplexers like 2:1, 4:1 and 8:1.

MUX :

Multiplexer is a combinational circuit that has maximum of 2n data inputs, 'n 'selection lines and single output line. One of these data inputs will be connected to the output based on the values of selection lines. A multiplexer in digital electronics is known as a data selector.

2:1 Multiplexer :

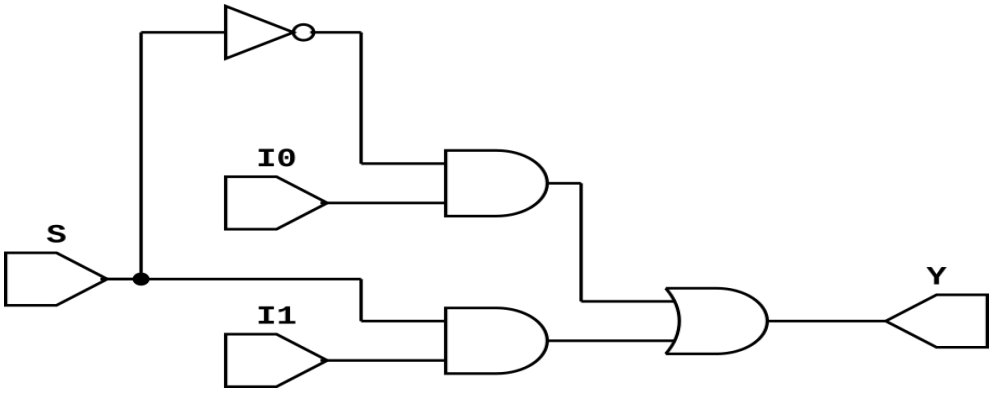
A 2-to-1 multiplexer consists of two inputs D0 and D1, one select input S and one output Y.Depending on the select signal, the output is connected to either of the inputs. Since there are two input signals, only two ways are possible to connect the inputs to the outputs, so one select is needed to do these operations.



Boolean Expression :

$Y = S'I_O + SI_1$

Logic Diagram :



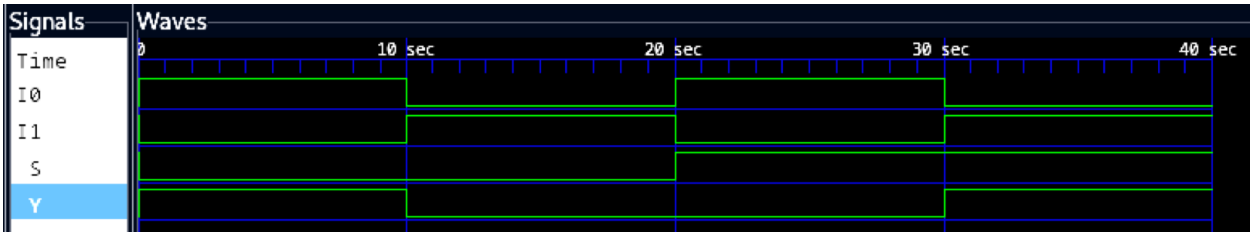
2:1 Multiplexer - Verilog Code :

```
module mux_2_to_1(I0,I1,S,Y);
input I0,I1,S;
output Y;
assign Y =(~S & I0)|(S & I1);
endmodule
```

Testbench Code:

```
module mux_2_to_1_tb;
reg I0,I1,S;
wire Y;
mux_2_to_1 mux_2_to_1_tb(I0,I1,S,Y);
initial
begin
S =0; I0 =1; I1 =0;
#10
S=0;I0 =0; I1 =1;
#10
S =1; I0 =1; I1 =0;
#10
S=1;I0 =0; I1 =1;
end
endmodule
```

Simulation waveform:



4x1 Multiplexer :

4x1 Multiplexer has four data inputs I_3, I_2, I_1 & I_0, two selection lines s_1 & s_0 and one output Y. The **block diagram** of 4x1 Multiplexer is shown in the following figure.

One of these 4 inputs will be connected to the output based on the combination of inputs present at these two selection lines.

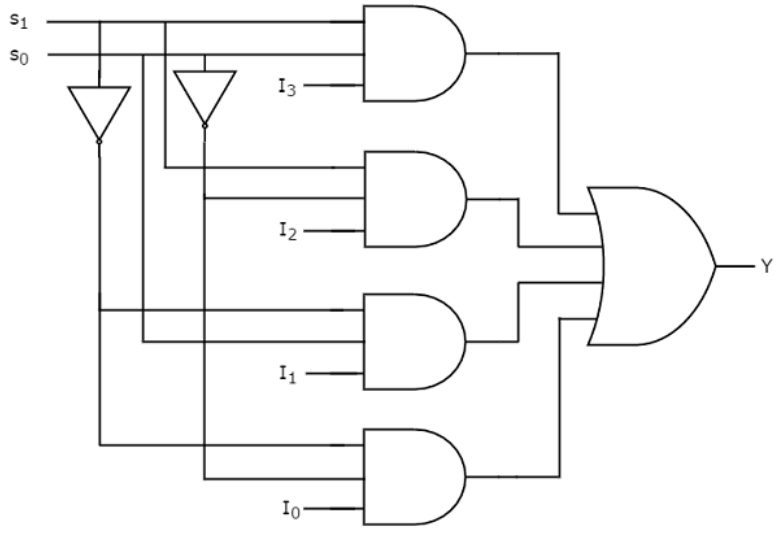
Truth table :

Selection Lines		Output
S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

From Truth table, we can directly write the **Boolean function** for output, Y as

$$Y = S_1'S_0'I_0 + S_1'S_0I_1 + S_1S_0'I_2 + S_1S_0I_3$$

We can implement this Boolean function using Inverters, AND gates & OR gate. The **circuit diagram** of 4x1 multiplexer is shown in the following figure.



4x1 Multiplexer - Verilog Code :

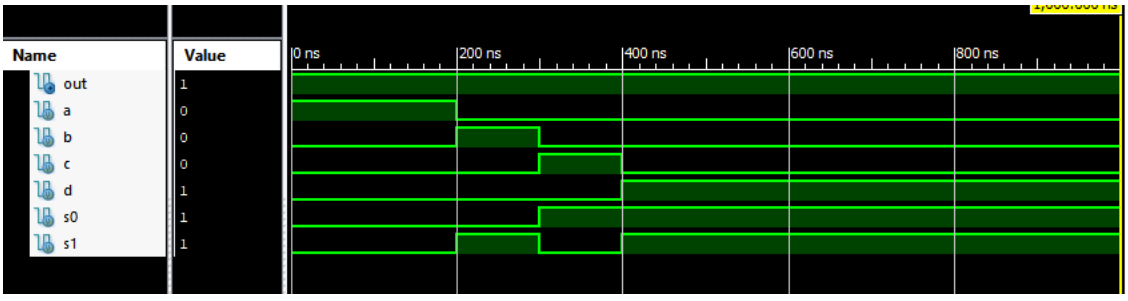
```
module mux_4_to_1(I0,I1,I2,I3,S0,S1,Y);
   input I0,I1,I2,I3,S0,S1;
   output Y;
   assign Y = (I0& (~S1) & (~S0)) |(I1& (~S1) & S0) |(I2 & S1 & (~S0)) |(I3& S1 & S0);
endmodule
```

Testbench Code:

```
module mux_4_to_1_tb;
reg I0,I1,I2,I3,S0,S1;
wire Y;
mux_4_to_1 mux_4_to_1_tb(I0,I1,I2,I3,S0,S1,Y);
initial
begin
S1 = 0; S0 = 0;  I0 = 1; I1 = 0; I2 = 0; I3 = 0;
#10
S1 = 0; S0 = 1;  I0 = 0; I1 = 1; I2 = 0; I3 = 0;
#10
S1 = 1; S0 = 0;  I0 = 1; I1 = 0; I2 = 1; I3 = 0;
#10
S1 = 1; S0 = 1;  I0 = 0; I1 = 0; I2 = 0; I3 = 1;
```

```
end
endmodule
```

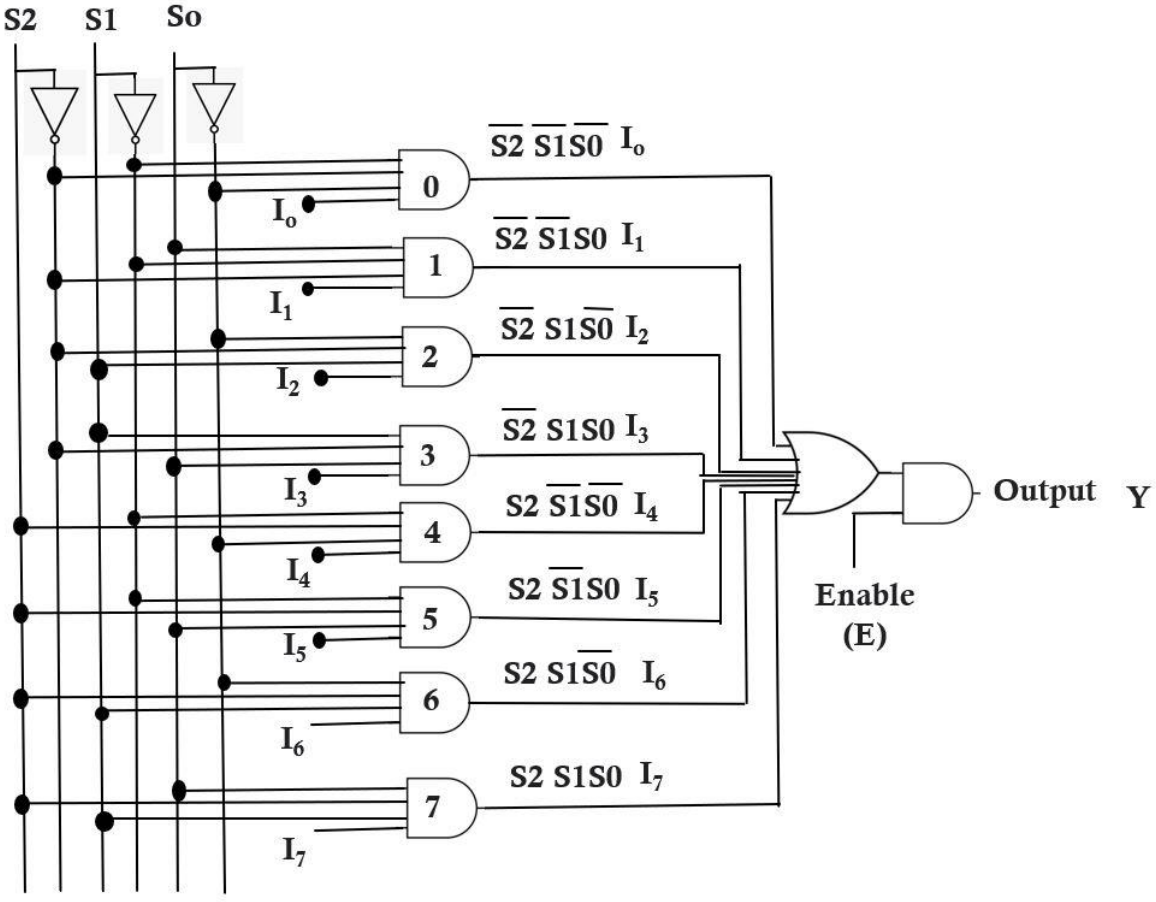
Simulation waveform:



8X1 Multiplexer :

Selection Inputs			Output
S_2	S_1	S_0	Y
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7

Out = S2' • S1' • S0' • I0 + S2' • S1' • S0 • I1 + S2' • S1 • S0' • I2 + S2' • S1 • S0 • I3 + S2 • S1' • S0' • I4 + S2 • S1' • S0 • I5 + S2 • S1 • S0' • I6 + S2 • S1 • S0 • I7



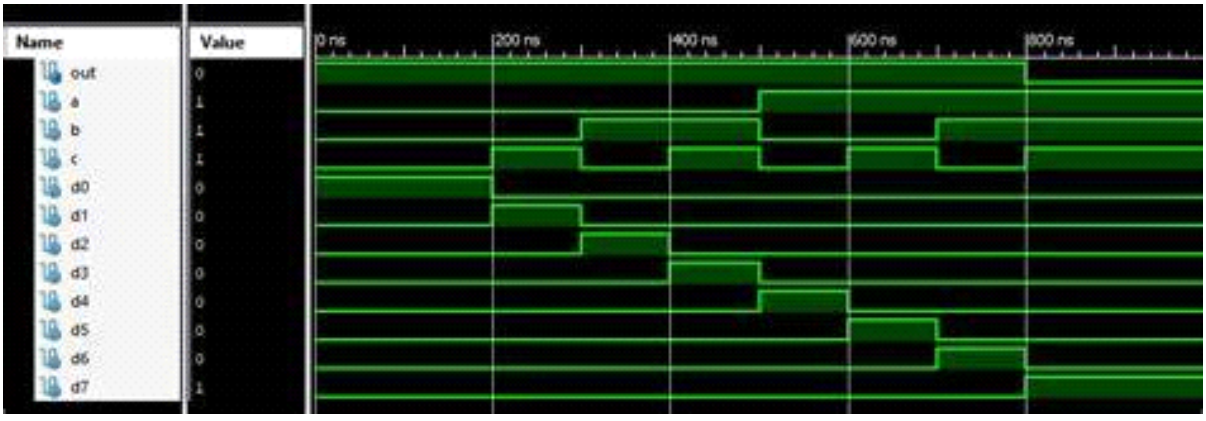
8X1 Multilexer - Verilog Code

```
module mux_8_to_1(I0,I1,I2,I3,I4,I5,I6,I7,S0,S1,S2,Y);
input I0,I1,I2,I3,I4,I5,I6,I7,S0,S1,S2;
output Y;
assign Y = (I0 & ~S2 & ~S1 & ~S0) | (I1 & ~S2 & ~S1 & S0) | (I2 & ~S2 & S1 & ~S0)
    | (I3 & ~S2 & S1 & S0) | (I4 & S2 & ~S1 & ~S0) | (I5 & S2 & ~S1 & S0) | (I6 & S2
    & S1 & ~S0) | (I7 & S2 & S1 & S0);
endmodule
```

Testbench Code:

```
module mux_8_to_1_tb;
reg I0,I1,I2,I3,I4,I5,I6,I7,S0,S1,S2;
wire Y;
mux_8_to_1 mux_8_to_1_tb(I0,I1,I2,I3,I4,I5,I6,I7,S0,S1,S2,Y);
initial
begin
S2 =0; S1 =0; S0 =0;  I0 =1; I1 =0; I2 =0; I3 =0; I4 =1; I5 =0; I6 =0; I7 =0;
#10
S2 =0; S1 =0; S0 =1;  I0 =0; I1 =1; I2 =0; I3 =0; I4 =1; I5 =0; I6 =0; I7 =0;
#10
S2 =0; S1 =1; S0 =0;  I0 =0; I1 =0; I2 =1; I3 =0; I4 =1; I5 =0; I6 =0; I7 =0;
#10
S2 =0; S1 =1; S0 =1;  I0 =0; I1 =0; I2 =0; I3 =1; I4 =0; I5 =0; I6 =0; I7 =0;
#10
S2 =1; S1 =0; S0 =0;  I0 =0; I1 =0; I2 =0; I3 =0; I4 =1; I5 =0; I6 =0; I7 =0;
#10
S2 =1; S1 =0; S0 =1;  I0 =0; I1 =0; I2 =0; I3 =0; I4 =0; I5 = 1; I6 =0; I7 =0;
#10
S2 =1; S1 =1; S0 =0;  I0 =0; I1 =0; I2 =0; I3 =0; I4 =0; I5 = 0; I6 =1; I7 =0;
#10
S2 =1; S1 =1; S0 =1;  I0 =0; I1 =0; I2 =0; I3 =0; I4 =0; I5 = 0; I6 =0; I7 =1;
end
endmodule
```

Simulation waveform:



Program 7

Design Verilog HDL to implement different types of De-multiplexers like 1:2, 1:4 and 1:8

1: 2 De-multiplexer:

In the 1 to 2 De-multiplexer, there are only two outputs, i.e., Y_0, and Y_1, 1 selection lines, i.e., S_0, and single input, i.e., A. On the basis of the selection value, the input will be connected to one of the outputs. The block diagram and the truth table of the 1×2 multiplexer are given below.

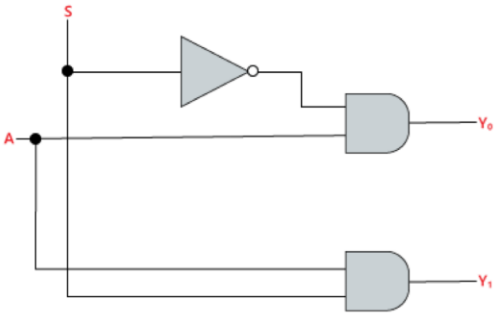
Truth Table:

INPUTS	Output	
S_0	Y_1	Y_0
0	0	A
1	A	0

The logical expression of the term Y is as follows:

$Y_0=S_0'.A$
$Y_1=S_0.A$

Logical circuit of the above expressions is given below:



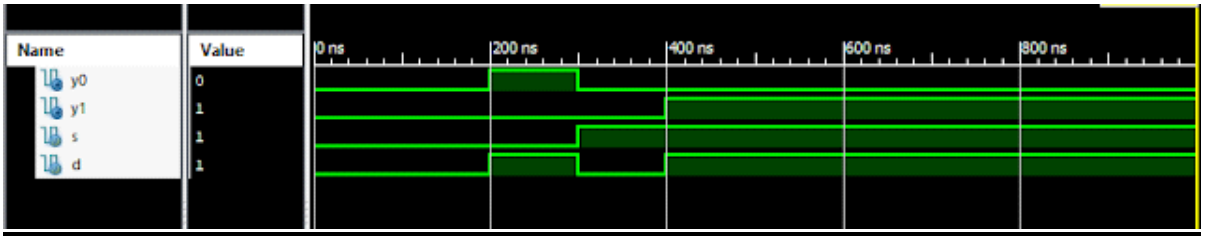
1:2 Demultiplexer Verilog Code:

```
module demux_1_to_2(I0,S,Y0,Y1);
input I0,S;
output Y0,Y1;
assign Y0 =(~S & I0);
assign Y1 =(S & I0);
endmodule
```

Testbench Code :

```
module demux_1_to_2_tb;
reg I0,S;
wire Y0,Y1;
demux_1_to_2 demux_1_to_1_tb(I0,S,Y0,Y1);
initial
begin
S = 0; I0 = 0;
#10
S=0;I0 = 1;
#10
S = 1; I0 = 0;
#10
S=1;I0 = 1;
end
endmodule
```

Simulation waveform:



1: 4 De-multiplexer:

1x4 De-Multiplexer has one input I, two selection lines, s_1 & s_0 and four outputs Y_3, Y_2, Y_1 &Y_0. The **block diagram** of 1x4 De-Multiplexer is shown in the following figure.

The single input 'I 'will be connected to one of the four outputs, Y_3 to Y_0 based on the values of selection lines s_1 & s0.

The **Truth table** of 1x4 De-Multiplexer is shown below.

Selection Inputs		Outputs			
S_1	S_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	**I**
0	1	0	0	**I**	0
1	0	0	**I**	0	0
1	1	**I**	0	0	0

From the above Truth table, we can directly write the **Boolean functions** for each output as

$$Y_3 = s_1 s_0 I$$

$$Y_2 = s_1 s_0' I$$

$$Y_1 = s_1' s_0 I$$

$$Y_0 = s_1' s_0' I$$

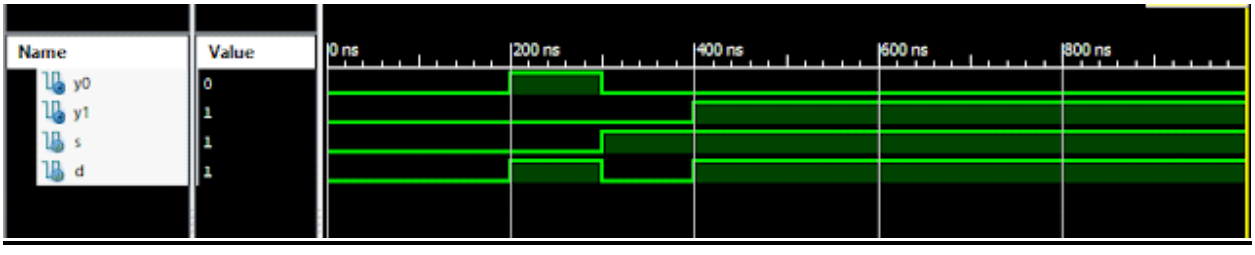
1:4 Demultiplexer Verilog Code:

```
module demux_1_to_4(I0,S1,S0,Y0,Y1,Y2,Y3);
input I0,S1,S0;
output Y0,Y1,Y2,Y3;
assign Y0 =(~S1 & I0 & ~S0);
assign Y1 =(S0 & I0 & ~S1);
assign Y2 =(~S0 & I0 & S1);
assign Y3 =(S0 & I0 & S1);
endmodule
```

Testbench Code :

```
module demux_1_to_4_tb;
reg S1,S0,I0;
wire Y0,Y1,Y2,Y3;
demux_1_to_4 demux_1_to_4_tb(I0,S1,S0,Y0,Y1,Y2,Y3);
initial
begin
I0 = 1;S0 = 0;S1 = 0;
#10;I0 = 1;S0 = 1;S1 = 0;
#10;I0 = 1;S0 = 0;S1 = 1;
#10;I0 = 1;S0 = 1;S1 = 1;
end
endmodule
```

Simulation waveform:



1:8 Demultiplexer :

In 1 to 8 De-multiplexer, there are total of eight outputs, i.e., Y_0, Y_1, Y_2, Y_3, Y_4, Y_5, Y_6, and Y_7, 3 selection lines, i.e., S_0, $S_1$and S_2 and single input, i.e., A. On the basis of the combination of inputs which are present at the selection lines S^0, S^1 and S_2, the input will be connected to one of these outputs. The block diagram and the truth table of the 1×8 de-multiplexer are given below.

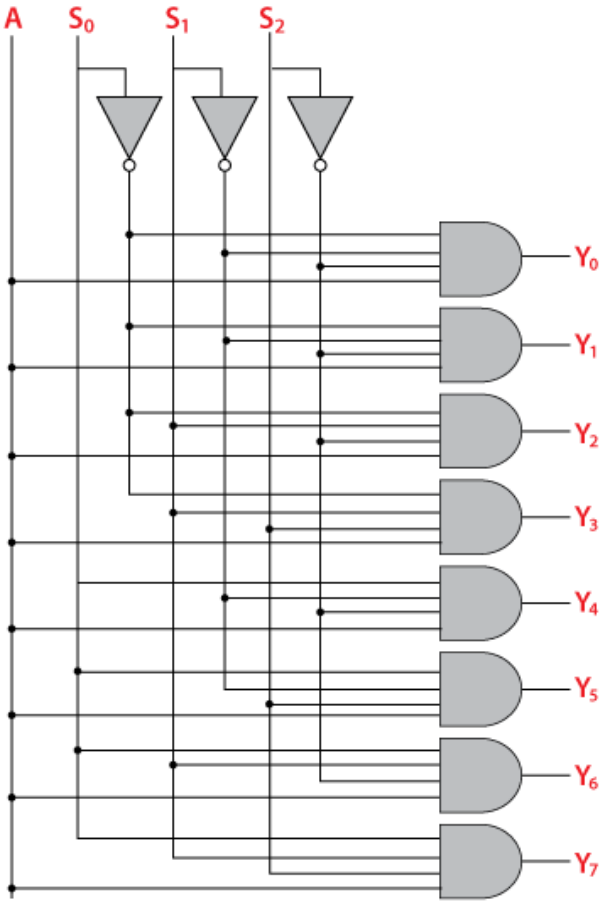
Truth Table:

INPUTS			Output							
S_2	S_1	S_0	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	0	A
0	0	1	0	0	0	0	0	0	A	0
0	1	0	0	0	0	0	0	A	0	0
0	1	1	0	0	0	0	A	0	0	0
1	0	0	0	0	0	A	0	0	0	0
1	0	1	0	0	A	0	0	0	0	0
1	1	0	0	A	0	0	0	0	0	0
1	1	1	A	0	0	0	0	0	0	0

The logical expression of the term Y is as follows:

$$Y_0=S_0'.S_1'.S_2'.A$$
$$Y_1=S_0.S_1'.S_2'.A$$
$$Y_2=S_0'.S_1.S_2'.A$$
$$Y_3=S_0.S_1.S_2'.A$$
$$Y_4=S_0'.S_1'.S_2\ A$$
$$Y_5=S_0.S_1'.S_2\ A$$
$$Y_6=S_0'.S_1.S_2\ A$$
$$Y_7=S_0.S_1.S_3.A$$

Logical circuit of the above expressions is given below:



1:8 Demultiplexer Verilog Code:

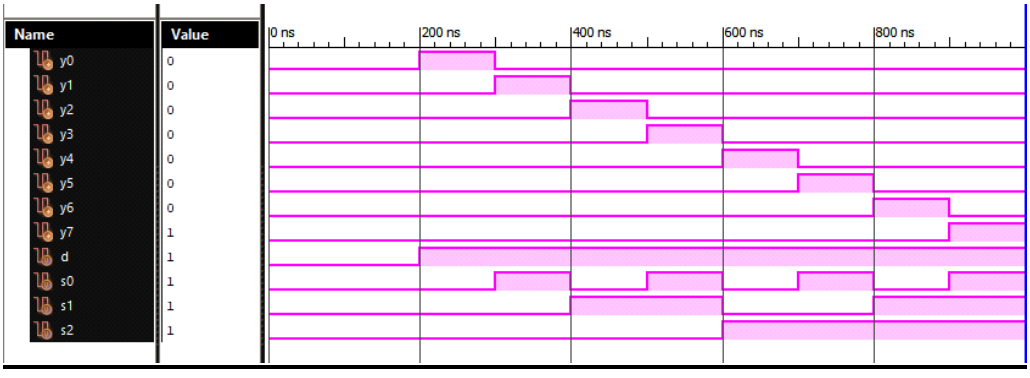
```
module demux_1_to_8(I0,S2,S1,S0,Y0,Y1,Y2,Y3,Y4,Y5,Y6,Y7);
input I0,S2,S1,S0;
output Y0,Y1,Y2,Y3,Y4,Y5,Y6,Y7;
assign Y0 =(~S2 & ~S1 & I0 & ~S0);
assign Y1 =(~S2 & S0 & I0 & ~S1);
assign Y2 =(~S2 & ~S0 & I0 & S1);
assign Y3 =(~S2 & S0 & I0 & S1);
assign Y4 =(S2 & ~S0 & I0 & ~S1);
```

```
assign Y5 =(S2 & S0 & I0 & ~S1);
assign Y6 =(S2 & ~S0 & I0 & S1);
assign Y7 =(S2 & S0 & I0 & S1);
endmodule
```

Testbench Code :

```
module demux_1_to_4_tb;
reg S2,S1,S0,I0;
wire Y0,Y1,Y2,Y3;
demux_1_to_4 demux_1_to_4_tb(I0,S2,S1,S0,Y0,Y1,Y2,Y3,Y4,Y5,Y6,Y7);
initial
begin
I0 = 1;S0 = 0;S1 = 0;S2 = 0;
#10;I0 = 1;S0 = 1;S1 = 0;S2 = 0;
#10;I0 = 1;S0 = 1;S1 = 1;S2 = 0;
#10;I0 = 1;S0 = 1;S1 = 1;S2 = 0;
#10;I0 = 1;S0 = 0;S1 = 0;S2 = 1;
#10;I0 = 1;S0 = 1;S1 = 0;S2 = 1;
#10;I0 = 1;S0 = 0;S1 = 1;S2 = 1;
#10;I0 = 1;S0 = 1;S1 = 1;S2 = 1;
end
endmodule
```

Simulation waveform:



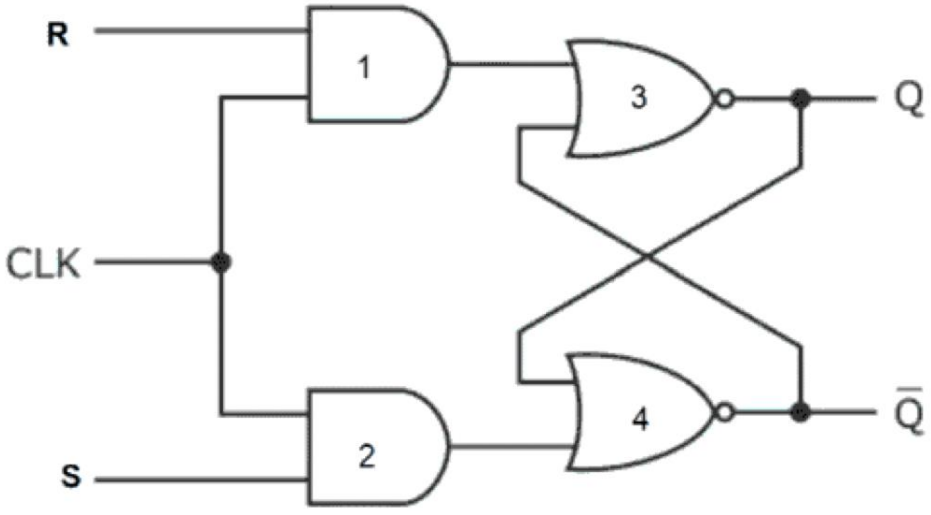
Program 8

Design Verilog program for implementing various types of Flip-Flops such as SR, JK and D

SR Flip-Flops :

This is the most common flip-flop among all. This simple flip-flop circuit has a set input (S) and a reset input (R). Apart from this, you can see the clock at the input end. At the output end, you will find two complementary outputs. The output is also feedback to the input visible in its circuit. Hence Q and Q ’are not the actual outputs.

You can make this simple flip flop either using AND and NOR gates or just NAND gates. We have taken the basic circuit of SR flip flop using AND and NOR gates for better understanding. You can check the logic gate basics.



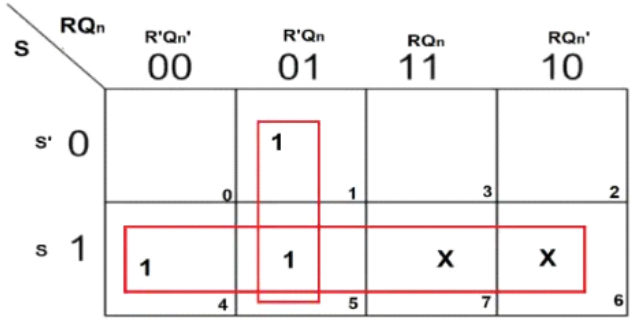
Let us take a look at the possible cases and write it down in our truth table. The clock is always 1.

SR Flip-Flop Truth Table

S	R	Q(n+1)	State
0	0	Qn	No change
0	1	0	RESET
1	0	1	SET
1	1	x	INVALID

Characteristic Table

S	R	Qn	Q(n+1)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	X
1	1	1	X



SR Flip Flop K-Map

From the K-map you get 2 pairs. On solving both we get the following characteristic equation:
Q(n+1) = S + R'Qn

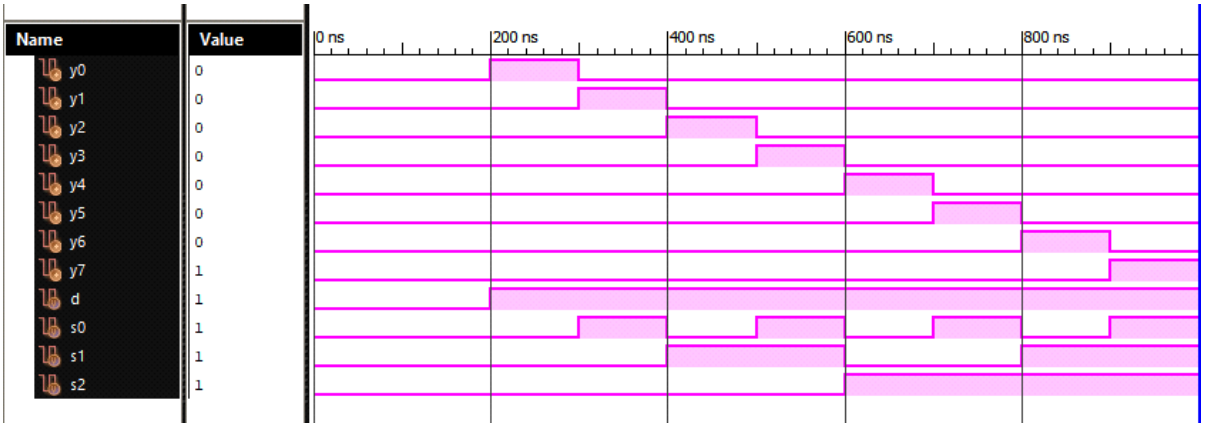
SR Flipflop - Verilog Code

```
module srff(s,r,clk,q,qb);
input s,r,clk;
inout q,qb;
wire s1,r1;
assign s1=!(s & clk);
assign r1=!(r&clk);
assign q=!(s1&qb);
assign qb=!(r1&q);
endmodule
```

Test Bench code :

```
module srff_tb;
reg S,R,CLK;
wire Q,Qbar;
srff srff_tb(S,R,CLK,Q,Qbar);
initial
begin
forever
begin
CLK=1;
#50 CLK=0;
#50 CLK=1;
end
end
initial
begin
S=0;R=1;
#10 S=0;R=0;
#10 S=1;R=0;
#10 S=1;R=1;
end
endmodule
```

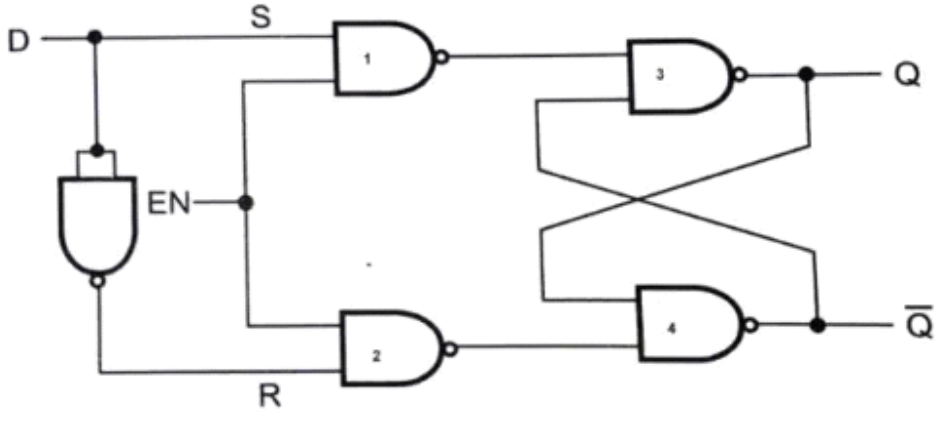
Simulation waveform:



D Flip-Flops :

D Flip flops or data flip flops or delay flip flops can be designed using SR flip flops by connecting a not gate in between S and R inputs and tying them together. D flip flops can be used in place of SR flip flops where you need only SET and RESET state.

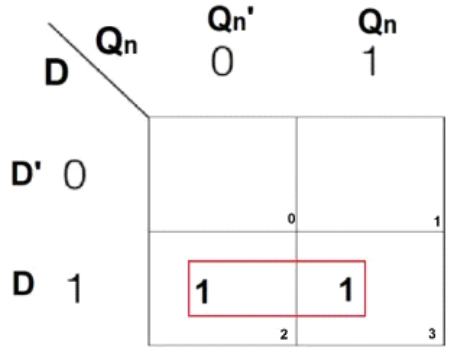
D Flip-Flop Circuit



D Flip-Flop Truth Table

CLK	D	Q(n+1)	State
–	0	0	RESET
–	1	1	SET

D	Qn	Q(n+1)
0	0	0
0	1	0
1	0	1
1	1	1



D Flip Flop K Map

From the K-map you get 2 pairs. On solving both we get the following characteristic equation:

```
Q(n+1) = D
```

D Flip Flop - Verilog Code :

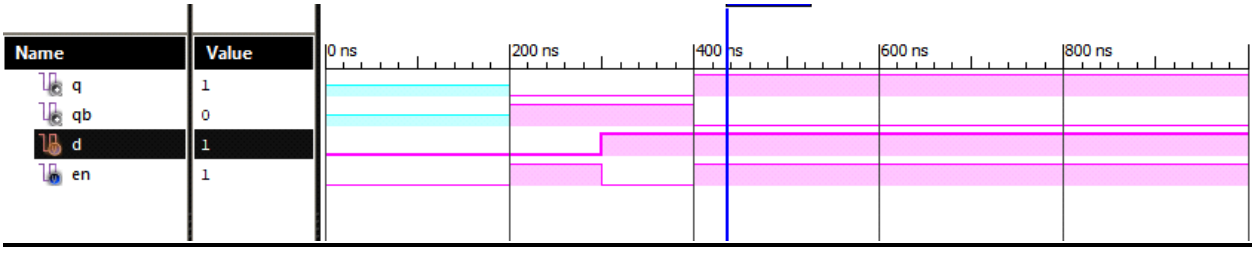
```
module dff(d,en,q,qb);
input d,en;
output q,qb;
wire a,b;
assign a = (en & ~d);
assign b = (en & d);
assign q = ~(a | qb);
```

```
assign qb = ~(b | q);
endmodule
```

Testbench Code :

```
module dff_tb;
reg d,en;
wire q,qb;
dff dff_tb(d,en,q,qb);
initial
begin
#10 d = 0;en = 1;
#10 d = 1;en = 0;
#10 d = 1;en = 1;
end
endmodule
```

Simulation waveform:



JK Flip Flop :

The "JK flip flop," also known as the Jack Kilby flip flop, is a sequential logic circuit designed by Jack Kilby during his tenure at Texas Instruments in the 1950s. This flip flop serves the purpose of storing and manipulating binary information within digital systems.

Basic Functionality of JK Flip Flop :

JK flip flop operates on sequential logic principle, where the output is dependent not only on the current inputs but also on the previous state. There are two inputs in JK Flip Flop Set and Reset denoted by J and K. It also has two outputs Output and complement of Output denoted by Q and $\overline{Q}$. The internal circuitry of a JK Flip Flop consists of a combination of logic gates, usually NAND gates.

JK flip flop comprises four possible combinations of inputs: J=0, K=0; J=0, K=1; J=1, K=0; and J=1, K=1. These input combinations determine the behavior of flip flop and its output.

J=0, K=0: In this state, flip flop retains its preceding state. It neither sets nor resets itself, making it stable.

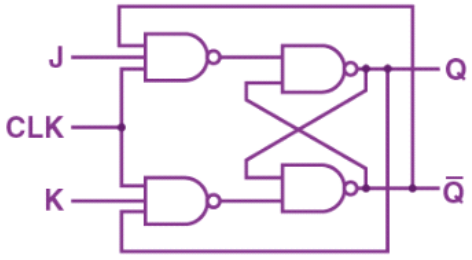
J=0, K=1: This input combination forces flip flop to reset, resulting in Q=0 and $\overline{Q}$=1. It is often referred to as the "reset" state.

J=1, K=0: Here, flip flop resides in the set mode, causing Q=1 and $\overline{Q}$=0. It is known as the "set" state.

J=1, K=1: This combination toggles flip flop. If the previous state is Q=0, it switches to Q=1 and vice versa. This makes it valuable for frequency division and data storage applications.

JK Flip Flop Truth Table :

JK flip flop truth table illustrates the relationship between the inputs (J and K) and the outputs (Q and $\overline{Q}$) of flip flop. Here is **JK flip flop truth table**:



Truth Table

J	K	Q_N	Q_{N+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Boolean Expression :

$Q_{n+1} = Q'_n (JK + JK') + Q_n (J'K' + JK')$

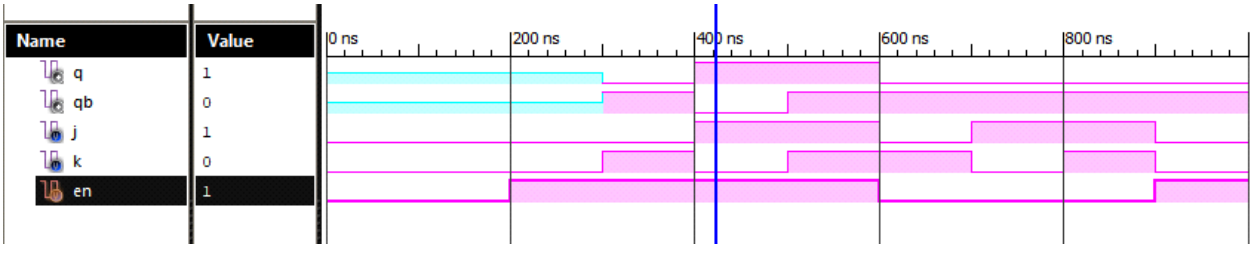
J K Flip Flop - Verilog Code :

```
module jkff(j,k,en,q,qb);
input j,k,en;
output q,qb;
assign a = ~qb & j & en;
assign b = ~(q & k & en);
assign q = ~(a & qb);
assign qb = ~(b & q & ~k);
endmodule
```

Testbench Code :

```
module jkff_tb;
reg J,K,CLK;
wire Q,Qbar;
jkff jkff_tb(J,K,CLK,Q,Qbar);
initial
begin
forever
begin
CLK=1;
#50 CLK=0;
#50 CLK=1;
end
end
initial
begin
J=0;K=1;
#10 J=0;K=0;
#10 J=1;K=0;
#10 J=1;K=1;
end
endmodule
```

Simulation waveform:



**********END**********

Content beyond Syllabus program

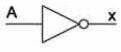
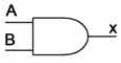
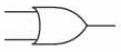
	Programs	CO
1	Basic Logic gate	CO1,CO2
2	Random Acess Memory	CO2
3	Program for arithmetic logic unit	CO3,CO4

Content beyond Syllabus program

Program-1

BASIC GATES :

Logic Gates

Name	NOT	AND	NAND	OR	NOR	XOR	XNOR
Alg. Expr.	$\overline{A}$	AB	$\overline{AB}$	$A+B$	$\overline{A+B}$	$A \oplus B$	$\overline{A \oplus B}$
Symbol							
Truth Table	A | X 0 | 1 1 | 0	B A | X 0 0 | 0 0 1 | 0 1 0 | 0 1 1 | 1	B A | X 0 0 | 1 0 1 | 1 1 0 | 1 1 1 | 0	B A | X 0 0 | 0 0 1 | 1 1 0 | 1 1 1 | 1	B A | X 0 0 | 1 0 1 | 0 1 0 | 0 1 1 | 0	B A | X 0 0 | 0 0 1 | 1 1 0 | 1 1 1 | 0	B A | X 0 0 | 1 0 1 | 0 1 0 | 0 1 1 | 1

Verilog code :

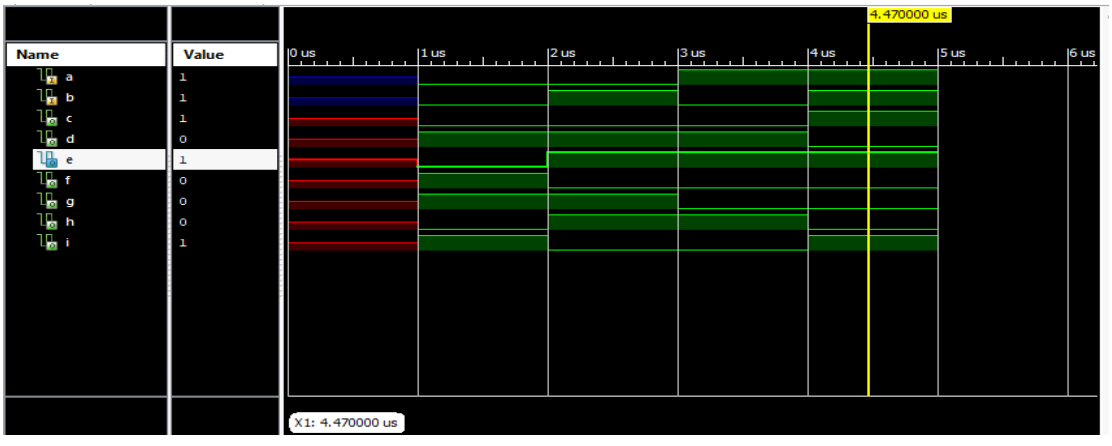
```
module allgates (a, b, c, d, e, f, g, h, i);
 input a,b;
 output c,d,e,f,g,h,i;
assign c= a&b;
assign d= ~(a&b);
assign e= a|b;
assign f= ~(a|b);
assign g= ~a;
```

```
assign h= a^b;
 assign i= ~(a^b);
end module
```

Testbench code :

```
module allgates_tb;
reg a,b;
wire c,d,e,f,g,h,i;
allgates allgates_tb(a,b,c,d,e,f,g,h,i);
initial
begin
a=0;b=0;
#15 a=0;b=1;
#15 a=1;b=0;
#15 a=1;b=1;
end
endmodule
```

Simulation Output :



Program-2

Verilog Program- Random Acess Memory

```
module ram(data,clk,reset,cs,we,addr,out);
input [3:0]
data,addr;
input clk,reset,cs,we;
output reg [3:0]
out;
reg [3:0]
ram[0:15];
always @(*)
begin
if(cs)
if(we)
begin
ram[addr]=data;
out=4'bzzzz;
end
else
out=ram[addr];
else
out=4'bzzzz;
end
endmodule
```

Program-3

Verilog code for ALU

```
module alu(
      input           [15:0]     a,           //src1
      input           [15:0]     b,           //src2
      input          [2:0]     alu_control,     //function
sel
      output    reg    [15:0]    result,         //result
      output zero
   );
 always @(*)  //fpga4student.com: FPga projects, Verilog
projects, VHDL projects
 begin
      case(alu_control)
      3'b000: result = a + b; // add
      3'b001: result = a - b; // sub
      3'b010: result = a & b; // and
      3'b011: result = a | b; // or
      3'b100: begin if (a<b) result = 16'd1;
                     else result = 16'd0;
                     end
      default:result = a + b; // add
      endcase
 end
 assign zero = (result==16'd0) ? 1'b1: 1'b0;
 endmodule
```

Viva question

1.What is the difference between Latch And Flip-flop?

The difference between latches and Flip-flop is that the latches are level triggered and flip-flops are edge triggered. In latches level triggered means that the output of the latches changes as we change the input and edg

e triggered means that control signal only changes its state when goesfrom low to high or high to low.

2. Name the two forms of Boolean expression?

The two forms of Boolean expression are:

1. Sum of products (SOP) form.
2. The Product of sum (POS) form.

3.What are Minterm and Maxterm?

A minterm is called Product of sum because they are the logical AND of the set of variables and Maxterm are called sum of product because they are the logical OR of the set of variables.

4.Write down the Characteristics of Digital ICs?

The characteristics of digital ICs are -

1. Propagation delay.
2. Power Dissipation.
3. Fan-in.
4. Fan-out.
5. Noise Margin.

5.What are the limitations of the Karnaugh Map?

The limitations of Karnaugh Map are as follows:

1. It is limited to six variable maps which means more than six variable involving expressions are not reduced.
2. These are useful for only simplifying Boolean expression which is represented I standard form

.

6.What are the advantages and disadvantages of the K-Map Method?

The advantages of the K-Map method are as follows-

1. It is an excellent method for simplifying expression up to four variables.
2. For the logical simplification, it gives us a visual method.
3. It is suitable for both SOP and POS forms of reduction.
4. It is more suitable for classroom teachings on logic simplification.

The disadvantages of the K-Map method are as follows:

1. It is not suitable when the number of variables exceeds more than four.
2. For Computer reduction, it is not suitable.
3. We have to take while entering the numbers in the cell-like 0, 1 and don't care terms.

7.Define Pair, Quad, and Octet?

Pair: Two adjacent cell of karnaugh map is called as Pair. It cancels one variable in a K-Map simplification.

Quad: A Pair of Four adjacent pairs in a karnaugh map is called a quad. It cancels two variables in a K-Map simplification.

Octet: A Pair of eight adjacent pair in a karnaugh map is called an octet. It cancels four variables in a K-map simplification.

8.Write the definition of the Duality Theorem?

Duality Theorem states that we can derive another Boolean expression with the existing Boolean expression by:

1. Changing OR operation (+ Sign) to AND operation (. Dot Sign) and vice versa.
2. Complimenting 0 and 1 in the expression by changing 0 to 1 and 1 to 0 respectively.

9. What are the applications of Multiplexer (MUX)?

The applications of the multiplexer are as follows:

1. It is used as a data selector from many inputs to get one output.
2. It is used as A/D to D/A Converter.

3. These are used in the data acquisition system.
4. These are used in time multiplexing system.

10. What are the applications of Demultiplexer?

The applications of the demultiplexer are as follows:

1. It is used in the data transmission system with error detection.
2. It is used as a decoder for the conversion of binary to decimal.
3. It is used as a serial to parallel converter.